



UNIVERSIDADE DE ÉVORA
ESCOLA DE CIÊNCIAS E TECNOLOGIA

DEPARTAMENTO DE INFORMÁTICA

DigitalWorks by Plexus Tech

Full-Stack Development

Licenciatura em Engenharia Informática

Estágio-Projeto 2025/2026



Miguel Rocha

Orientadores na Empresa: Ricardo Godinho; Renan de Medeiros

Orientador no Departamento: Lígia Ferreira

Trabalho desenvolvido na empresa DigitalWorks, no âmbito da disciplina de Estágio-Projeto da Licenciatura em Eng. Informática.

Évora, 19 de junho de 2026

Tabela de conteúdos

1	Introdução	2
1.1	Enquadramento institucional	2
1.2	Objetivos	2
1.3	Contribuições	2
1.4	Estrutura do documento	3
2	Ambiente de Trabalho	4
2.1	Integração na Empresa	4
2.2	Regime de Trabalho e Acompanhamento	5
3	Fundamentação Teórica	6
3.1	Arquitetura de aplicações web e separação de responsabilidades	6
3.2	Domain-Driven Design (DDD) e princípios SOLID	6
3.3	Persistência de dados e acesso à base de dados	7
3.4	Segurança em aplicações web	7
4	Contexto Operacional	9
4.1	Ambiente tecnológico	9
4.1.1	Hardware	9
4.1.2	Software e tecnologias utilizadas	9
4.1.3	Ambiente aplicacional	11
4.2	Metodologia de trabalho	11
5	Trabalho desenvolvido	13
5.1	Formação e Pequenos Projetos	13
5.1.1	Introdução a C#, .NET, MVC e APIs REST	13
5.1.2	Desenvolvimento com Umbraco CMS	14
5.1.3	Integração com DataTables e automatização de tarefas	16
5.1.4	Desenvolvimento de uma extensão para o Umbraco	17
5.2	Projeto Final	18
5.2.1	Planeamento inicial, princípios SOLID e arquitetura	18
5.2.2	Camadas <i>Domain</i> e <i>Infrastructure</i>	19
5.2.3	Camada <i>Application</i> e Autenticação	19
5.2.4	Camada <i>API</i>	20
5.2.5	Camada <i>Presentation</i>	21
5.2.6	Testes finais, correções e validação	22
6	Avaliação crítica	23
	Referências	24
	Anexo A – Imagens do projeto final	27

1 Introdução

O presente relatório foi elaborado no âmbito da unidade curricular Estágio-Projeto da Licenciatura em Engenharia Informática na Universidade de Évora, no ano letivo 2025/2026. O documento descreve as atividades e o trabalho realizados durante o estágio curricular na empresa DigitalWorks.

1.1 Enquadramento institucional

O estágio curricular teve início em 9 de fevereiro de 2026 e terminou em 11 de junho de 2026, totalizando 240 horas. Este foi realizado na empresa DigitalWorks, no seu escritório situado no Parque de Ciência e Tecnologia do Alentejo (PACT), em Évora.

Como não participei em nenhum projeto da empresa, não fui integrado numa equipa. No entanto, como a empresa tinha outro estagiário na mesma vaga, o Tiago Ramalho, trabalhamos em equipa constantemente durante o estágio, ajudando-nos mutuamente.

1.2 Objetivos

O principal objetivo do estágio foi adquirir conhecimentos na área de desenvolvimento de *software*, com maior incidência no *backend*, embora também tenham sido abordadas tecnologias de *frontend*. Para além disso, outro objetivo bastante importante foi a aquisição de conhecimentos sobre princípios e boas práticas, especialmente na programação orientada a objetos.

Inicialmente, um dos objetivos definidos na proposta de estágio consistia em participar num projeto real da empresa. Contudo, durante o estágio, foi-nos comunicado que essa participação já não seria possível. Esta alteração ao plano inicial permitiu abordar outras áreas relevantes, nomeadamente a automatização de tarefas e a utilização de APIs externas.

Como alternativa, o orientador na empresa propôs a realização de um projeto em grupo, a ser desenvolvido em conjunto com o meu colega de estágio. Este projeto teve como objetivo criar uma aplicação web simples de entrega de comida ao domicílio, a fim de aprender, consolidar e aplicar os conhecimentos adquiridos ao longo do estágio.

1.3 Contribuições

Como o trabalho desenvolvido não foi integrado num projeto real da empresa nem destinado à utilização por um cliente, não houve contribuição direta para a empresa nem para entidades externas.

No entanto, durante o estágio foi implementada uma extensão para o gestor de conteúdos Umbraco, cujo objetivo era centralizar os dados enviados por formulários do *website* numa tabela, de modo a carregar apenas uma página por vez. Desta forma, foi possível melhorar o desempenho do gestor de conteúdos, pois, atualmente, todos os dados são carregados para a construção da tabela. Este foi um problema relatado por um cliente da empresa, mas a solução implementada não chegou a ser utilizada por terem usado outra solução.

Para além disso, o estágio teve um contributo significativo para a minha aprendizagem e para o desenvolvimento de competências técnicas e pessoais.

1.4 Estrutura do documento

A presente secção apresentou o estágio em si, incluindo uma breve apresentação da instituição onde fui acolhido, os objetivos de estágio e as minhas principais contribuições.

Este documento ainda contém mais cinco secções. Na próxima irei descrever o ambiente no qual realizei o estágio, apresentando não só a empresa, como o meu processo de integração e o acompanhamento que fui recebendo. Posteriormente, apresentarei os principais conteúdos teóricos e tecnológicos associados ao meu trabalho. Após isso, são descritas as condições de *hardware* em que trabalhei, bem como as de *software* e as principais tecnologias utilizadas. Ainda são descritos o ambiente aplicacional e as metodologias utilizadas no trabalho. Em seguida, serão apresentados os trabalhos desenvolvidos ao longo do estágio, uma avaliação crítica de todo esse período e alguns dos resultados do trabalho desenvolvido.

2 Ambiente de Trabalho

A DigitalWorks é uma empresa tecnológica especializada no desenvolvimento de *software* e na transformação digital de organizações. A sua atividade abrange o desenvolvimento de aplicações para dispositivos móveis e para a web, integrações de nível empresarial e soluções de *Enterprise Resource Planning* (ERP) e de *Customer Relationship Management* (CRM) [1].

Esta empresa foi fundada em Lisboa em 2010 e, posteriormente, alargou a sua atividade ao Reino Unido, com a abertura de um escritório em Londres. Em Portugal, a empresa possui escritórios em Évora e em Lisboa [1].



Figura 1: Logotipo da DigitalWorks

Atualmente, a DigitalWorks encontra-se integrada no grupo empresarial Plexus Tech. Esta integração assegurou a continuidade dos serviços prestados pela DigitalWorks e passou a permitir o acesso às restantes soluções tecnológicas disponibilizadas pela Plexus Tech [2].

A Plexus Tech é uma empresa espanhola especializada na prestação de serviços e no desenvolvimento de produtos no setor de Tecnologias da Informação (TI). A sua sede está localizada em Santiago de Compostela, na Galiza, Espanha, e possui uma grande presença no território espanhol e em vários mercados internacionais [3]. Para além de Espanha e Portugal, a organização possui presença em países como o Reino Unido, a Alemanha, os Estados Unidos da América, o México, o Chile e a Colômbia [4].



Figura 2: Logotipo da Plexus Tech

2.1 Integração na Empresa

O primeiro dia de estágio foi dedicado ao processo de *onboarding*. Neste, eu e os restantes estagiários fomos recebidos numa sala de reuniões do escritório, onde foi realizada uma apresentação da

DigitalWorks e da Plexus Tech. Durante esta sessão, foram abordados diversos assuntos de ambas as empresas, como as respetivas áreas de atividade, os setores em que operavam, a organização interna, as parcerias e alguns dos resultados obtidos nos anos anteriores.

Esta sessão permitiu obter uma visão geral do funcionamento de ambas as empresas e foi bastante simples e aberta a perguntas.

Neste primeiro dia, também foi possível conhecer os restantes estagiários da empresa: três colegas da Universidade de Évora do estágio em *Mobile Development*, o meu colega de estágio, também da Universidade de Évora, e um colega de curso profissional da Escola Secundária Gabriel Pereira.

2.2 Regime de Trabalho e Acompanhamento

O estágio decorreu sob um regime de trabalho híbrido, que permitia a realização das atividades tanto presencialmente no escritório como remotamente. O horário de trabalho estava compreendido entre as 9h00 e as 18h00, com direito a um descanso de trinta minutos e a uma pausa para o almoço de uma hora.

Durante a maior parte do estágio, optei por trabalhar remotamente, uma vez que se revelou mais conveniente não só para mim como também para o meu colega de estágio.

As sessões de acompanhamento do estágio foram realizadas conforme as necessidades identificadas ao longo do período. Em algumas ocasiões, estas sessões eram solicitadas por mim e pelo meu colega Tiago, sobretudo quando era necessário esclarecer dúvidas, pedir ajuda, apresentar o trabalho realizado ou discutir o próximo trabalho a realizar. Em outras ocasiões, era o orientador da empresa que nos orientava na formação, Ricardo Godinho, ou o orientador que nos ajudava no trabalho final, Renan de Medeiros. Estas sessões tinham como objetivo obter *feedback*, acompanhar o desenvolvimento e, sobretudo, ajudar a desenvolver as tarefas.

3 Fundamentação Teórica

Durante o estágio, as atividades realizadas consistiram majoritariamente em formações e tutoriais sobre o desenvolvimento de aplicações *web* com recurso à plataforma .NET. Neste capítulo são abordados conceitos relacionados com a arquitetura de aplicações *web*, a separação de responsabilidades, o padrão *Model View Controller* (MVC) e a organização em camadas. São também introduzidos princípios de estruturação de código como SOLID e o *Domain-Driven Design* (DDD), bem como conceitos associados à persistência de dados, ao acesso a bases de dados e à segurança em aplicações *web*.

3.1 Arquitetura de aplicações web e separação de responsabilidades

As aplicações *web* são normalmente desenvolvidas num modelo cliente-servidor, no qual o cliente é responsável por apresentar uma interface ao utilizador e enviar pedidos, enquanto o servidor processa esses pedidos, aplica a lógica necessária e devolve uma resposta. Esta comunicação é normalmente feita através do protocolo HTTP, que constitui a base da troca de informação na *web* [5].

Este modelo esteve presente nos projetos desenvolvidos com ASP.NET Core, em particular no primeiro pequeno projeto descrito na subsecção 5.1.1, nos quais o *frontend* comunicava com o *backend* através de pedidos HTTP.

Um dos padrões utilizados no desenvolvimento de aplicações *web* é o MVC, que faz uma divisão de responsabilidades de forma a facilitar a organização e manutenção do código [6]. Segundo este padrão, a aplicação deve ser dividida em três componentes distintos:

- *Model*, que representa os dados e a lógica que lhe está associada.
- *View*, responsável pela apresentação da informação.
- *Controller*, que recebe os pedidos e coordena as respostas.

Para além de MVC, as aplicações de maior dimensão podem também beneficiar de uma organização por camadas. Esta abordagem está relacionada com os princípios de *Clean Architecture*, em que cada parte da aplicação possui uma responsabilidade específica, sendo comum separar a lógica do domínio, o acesso à base de dados, a API, a interface de apresentação, entre outros. Esta organização reduz a dependência entre componentes e facilita possíveis alterações futuras. Assim, modificações numa determinada camada têm um impacto muito reduzido nas restantes [7].

No projeto final, abordado na subsecção 5.2, a separação aplicada foi inspirada em *Clean Architecture*, com camadas como *Domain*, *Application*, *Infrastructure*, *API* e *Presentation*. Esta estrutura permitiu separar a lógica de "negócio", o acesso à base de dados, os serviços disponibilizados pela API e as interfaces utilizadas pelos clientes e fornecedores, tornando a solução mais organizada e adequada à aplicação de boas práticas de desenvolvimento.

3.2 Domain-Driven Design (DDD) e princípios SOLID

O DDD é uma abordagem de desenvolvimento de *software* focada na modelação do domínio da aplicação, isto é, na representação dos conceitos, regras e processos associados ao problema que o sistema

vai resolver. Em vez de organizar o código apenas em função de detalhes técnicos, esta abordagem procura aproximar a estrutura da aplicação dos conceitos reais que se pretendem representar [8].

No contexto do projeto final, esta ideia foi aplicada na utilização de uma camada *Domain*, onde foram representadas entidades associadas ao funcionamento da aplicação. Estas entidades representavam conceitos centrais do domínio da aplicação e permitiam concentrar a parte lógica e as regras relacionadas com esses conceitos, separando-as de coisas como as bases de dados ou a interface de utilizador.

Para além desta modelação de domínio, também tivemos em consideração os princípios SOLID, que representam um conjunto de boas práticas para o desenvolvimento em *Object-Oriented Programming* (OOP). SOLID corresponde aos princípios *Single Responsibility*, *Open/Closed*, *Liskov Substitution*, *Interface Segregation* e *Dependency Inversion*, que procuram tornar o código mais organizado, flexível e fácil de manter. Estes princípios incentivam a separação de responsabilidades, a redução de dependência entre classes e mesmo a dependência de abstrações em vez de implementações concretas [9].

Nos trabalhos descritos na secção 5, estes princípios foram aplicados nos projetos desenvolvidos, de forma a organizar melhor as responsabilidades entre as classes e camadas.

3.3 Persistência de dados e acesso à base de dados

A persistência de dados é um conceito essencial no desenvolvimento de aplicações, uma vez que permite armazenar informação de forma persistente para que esta possa ser consultada, alterada ou até mesmo removida. Em aplicações *web*, esta persistência é frequentemente relacionada através de bases de dados relacionais, onde a informação é organizada em tabelas, com linhas e colunas, e relacionada através de chaves [10].

Existem diferentes formas de realizar o acesso a dados numa aplicação. Uma abordagem comum no ecossistema .NET é a utilização de um *Object-Relational Mapping* (ORM). O Entity Framework Core permite trabalhar com bases de dados através de objetos .NET, reduzindo a necessidade de escrever manualmente grande parte do código que acede aos dados [11]. Esta abordagem simplifica o desenvolvimento, mas também abstrai parte da comunicação com a base de dados.

Outra abordagem consiste no acesso aos dados através de ADO.NET. Esta abordagem exige maior controlo sobre as instruções SQL, os parâmetros, a leitura dos resultados e o *mapping* entre objetos da aplicação e objetos da base de dados [12].

No projeto final, foi utilizada esta segunda abordagem, o que tornou necessário compreender melhor a criação de comandos SQL, a abertura de conexões e o *mapping* manual dos dados. Também foram relevantes as *transactions*, usadas para garantir a consistência dos dados quando uma operação envolvia várias alterações para fazer um *rollback* dos dados, caso uma operação falhasse [13].

3.4 Segurança em aplicações web

A segurança em aplicações *web* assenta em mecanismos de autenticação e autorização que permitem, respetivamente, identificar utilizadores e controlar o acesso a recursos e operações do sistema. A autenticação consiste no processo de validação da identidade de um utilizador com base em credenciais.

A autorização determina os privilégios e permissões atribuídos a um utilizador autenticado dentro da aplicação.

Uma abordagem comum em APIs consiste na utilização de *tokens*. O *JSON Web Token* (JWT) é um formato compacto para representar *claims*, ou seja, declarações sobre uma entidade, como o identificador do utilizador, o seu papel ou outros atributos relevantes [14]. Estes *tokens* podem ser enviados pelo cliente nos pedidos seguintes, nomeadamente no cabeçalho *Authorization*, utilizando o esquema Bearer, permitindo que a API valide a identidade do utilizador autenticado [15].

Os JWT incluem um conjunto de *claims* que podem ser classificados como registadas, públicas ou privadas. Entre as mais relevantes destacam-se *sub* (identificador do sujeito), *exp* (data de expiração), *iat* (data de emissão) e *role* ou *roles*, frequentemente utilizadas para suporte a mecanismos de controlo de acesso baseado em perfis. Em ASP.NET Core, esta abordagem permite implementar autorização baseada em *roles*, restringindo o acesso a determinadas funcionalidades com base no perfil do utilizador [16].

Para além do *access token*, utilizado na autenticação de pedidos às APIs, é comum a utilização de *refresh tokens*. Estes permitem a renovação do *access token* após a sua expiração sem a necessidade de uma nova autenticação do utilizador, desde que o refresh token seja válido e não esteja revogado. Ao utilizar *access tokens* de curta duração, existe, assim, uma melhoria na experiência de utilização [17].

4 Contexto Operacional

4.1 Ambiente tecnológico

4.1.1 Hardware

O computador que recebi para o estágio foi um portátil ThinkPad E15 Gen 4, com um processador Intel Core i5-1235U, 16 GB de memória RAM e um SSD de 256 GB. Este computador foi suficiente para realizar as tarefas do estágio sem problemas.

4.1.2 Software e tecnologias utilizadas

Durante a realização do estágio, foram utilizados diferentes *softwares*, que serviram para desenvolver as tarefas propostas e cumprir as práticas internas da empresa. As principais ferramentas e *softwares* utilizados foram:

- **Windows** – O Windows foi o sistema operativo utilizado no computador disponibilizado durante o estágio. Neste sistema, foram utilizadas as ferramentas de desenvolvimento e executados os serviços e as aplicações necessários à realização das tarefas [18].
- **Microsoft Visual Studio** – É um *Integrated Development Environment* (IDE) e foi utilizado para organizar os projetos, criar, editar, compilar e executar as aplicações desenvolvidas.
- **C#** – É uma linguagem de programação orientada a objetos. Foi a principal linguagem de programação utilizada em todos os projetos realizados [19].
- **.NET SDK** – O .NET SDK corresponde ao conjunto de ferramentas e componentes necessários para criar, compilar, testar, executar e gerir aplicações desenvolvidas na plataforma .NET [20]. Este foi utilizado em todos os projetos desenvolvidos com .NET, incluindo APIs, aplicações MVC e Blazor.
- **ASP.NET Core** – É um *framework* utilizado no desenvolvimento de aplicações *web* e de APIs. Este framework permitiu implementar *controllers*, serviços, rotas e, principalmente, a comunicação entre o *frontend* e o *backend* [21].
- **Entity Framework Core** – É um ORM utilizado para mapear classes C# para tabelas de uma base de dados relacional e facilitar as operações *Create*, *Read*, *Update* e *Delete* (CRUD) [11]. Foi utilizado principalmente nos primeiros projetos realizados, por permitir trabalhar com os dados através de objetos C# e reduzir a necessidade de escrever comandos SQL manualmente.
- **SQL Server** – O SQL Server é um sistema de gestão de base de dados relacionais utilizado para a persistência de dados [22]. Este permitiu armazenar os dados das aplicações desenvolvidas e garantir a respetiva consulta e alteração.
- **SQL Server Management Studio** – É uma ferramenta para gerir e consultar bases de dados SQL Server [23]. Através desta ferramenta foi possível visualizar tabelas, executar instruções SQL e criar as bases de dados dos diversos projetos.
- **Umbraco CMS** – O Umbraco CMS é um *Content Management System* (CMS) *open source* para

desenvolvimento em .NET [24]. Foi a ferramenta principal para desenvolver o *frontend* de *websites*, permitindo criar *websites* dinâmicos, configurar templates e páginas e personalizar o *backoffice*.

- **Blazor WebAssembly** – O Blazor WebAssembly foi utilizado no desenvolvimento da camada *presentation* do projeto final. Esta tecnologia permite criar interfaces e aplicações em C#, bem como executar os respetivos componentes no *browser* [25] [26].
- **Razor** – O Razor é uma sintaxe utilizada para combinar código C# com elementos *HyperText Markup Language* (HTML) para criar *views* e templates dinâmicos [27]. Durante o estágio, foi utilizada nos primeiros projetos, a fim de criar páginas e componentes capazes de exibir dados.
- **Bootstrap** – É um *framework* CSS que disponibiliza estilos por classes e componentes reutilizáveis que facilitam o desenvolvimento de um *layout* fluido para diversos tipos de ecrãs [28]. Foi utilizado no *frontend* dos primeiros projetos.
- **MudBlazor** – É uma biblioteca de componentes *User Interface* (UI) para desenvolvimento em Blazor [29]. No projeto final, foi utilizado para construir formulários, tabelas e outros componentes visuais, reduzindo a necessidade de desenvolver cada componente de raiz.
- **Postman** – O Postman é uma ferramenta utilizada para desenvolver e testar APIs [30]. Este foi usado para realizar pedidos HTTP a APIs, validar os *endpoints* desenvolvidos e analisar os conteúdos das respostas.
- **Swagger/OpenAPI** – o Swagger é uma ferramenta utilizada para testar e documentar APIs [31] [32]. Tal como o Postman, este também serviu para testar APIs, mas teve um papel importante em documentá-las.
- **DataTables** – É uma biblioteca para criar e melhorar tabelas. Ao usar esta biblioteca, foi possível mostrar dados retornados por APIs, com a possibilidade de pesquisar, ordenar, filtrar e paginar [33].
- **Hangfire** – É uma biblioteca para aplicações .NET e .NET Core para executar tarefas em segundo plano, tarefas recorrentes ou agendadas. Foi explorada num projeto intermédio para realizar pedidos periódicos à API [34].
- **Windows Task Scheduler** – É uma ferramenta do Windows que permite agendar a execução automática de programas ou *scripts* [35]. Foi utilizada como alternativa ao HangFire, para analisar outras opções e ter um critério de comparação.
- **Git e GitHub** – O Git foi utilizado para o controlo de versões do projeto final, de modo a registar e acompanhar as alterações feitas durante o desenvolvimento [36]. O GitHub foi utilizado maioritariamente para armazenar os projetos, abrir *pull requests* (PRs) e apoiar a partilha e colaboração durante o desenvolvimento dos mesmos [37].
- **Microsoft Edge e Mozilla Firefox** – Foram os *browsers* utilizados para consultar documentação, seguir tutoriais, bem como testar e validar as aplicações *web* desenvolvidas [38] [39].
- **NuGet** – O NuGet é um gestor de pacotes da plataforma .NET, utilizado para adicionar e gerir dependências dos projetos [40].
- **Plextime** – O Plextime é uma plataforma da PlexusTech para registar o tempo de trabalho du-

rante o estágio [41]. Através desta, foi possível registar diariamente as horas de entrada e de saída do dia de trabalho.

- **Redmine** – O Redmine é uma ferramenta que serve para alocar as horas diárias trabalhadas em projetos [42]. Desta forma, foi utilizada para consultar as horas restantes de trabalho para o fim de estágio, bem como para marcá-las.

4.1.3 Ambiente aplicacional

Os projetos realizados não fizeram parte de nenhum sistema empresarial em produção, tendo sido desenvolvidos como aplicações independentes, com o objetivo de consolidar os conhecimentos lecionados ao longo do curso e os obtidos durante o período de estágio, simulando cenários reais de desenvolvimento.

Numa fase inicial, foram desenvolvidos projetos *standalone* de menor dimensão, destinados à aprendizagem de conceitos fundamentais, como aplicações ASP.NET Core MVC, APIs REST, persistência de dados e consumo de serviços HTTP. Estes projetos permitiram compreender o funcionamento de aplicações *web*, a comunicação entre cliente e servidor e a interação com base de dados.

Foram também desenvolvidos projetos com o Umbraco CMS, cujo ambiente operacional consistiu na criação de *websites* geridos através de um *backoffice*. Nestes projetos, o conteúdo era criado e configurado no próprio CMS, sendo posteriormente apresentado através de templates Razor.

O projeto final, destinado a clientes que encomendam comida e aos próprios restaurantes, constituiu a aplicação mais completa desenvolvida durante o estágio. Trata-se de uma aplicação *standalone* realizada com uma arquitetura em camadas, *Clean Architecture*, composta por uma API, uma aplicação cliente e uma aplicação para fornecedores. A API disponibiliza os serviços principais da aplicação, enquanto os *frontends* comunicam com esta através de pedidos HTTP para que os dados sejam tratados posteriormente pelo *backend*.

Neste ambiente, a API interage com uma base de dados SQL Server para persistir informações relativas a utilizadores, produtos, carrinhos de compras, pagamentos, encomendas, moradas, entre outros. A aplicação do cliente permite criar e registar contas, alterar o perfil, consultar produtos, gerir o carrinho de compras e efetuar encomendas. Já a aplicação do fornecedor permite criar e registar contas, bem como gerir produtos, perfis, encomendas e stock.

Além disso, alguns projetos interagiram com outros sistemas ou componentes, como *scripts* e tarefas em segundo plano para consumir APIs e automatizar operações.

4.2 Metodologia de trabalho

Durante o estágio, o trabalho foi desenvolvido de forma progressiva e incremental, acompanhando a evolução da aprendizagem e a dificuldade dos projetos realizados. Estes trabalhos, apesar de não terem seguido nenhuma metodologia formal, eram construídos por etapas, normalmente com foco numa funcionalidade ou numa camada de cada vez. Eram sempre testados antes de passarem para a próxima etapa e, quando surgia algum erro, voltava-se atrás para corrigi-lo.

Em alguns momentos, algumas tarefas foram divididas, mas grande parte do desenvolvimento foi realizada em conjunto com o meu colega de estágio. Esta forma de trabalho foi a mais eficaz para ambos, pois permitiu discutir soluções, esclarecer dúvidas e garantir que ambos acompanhassem a evolução dos projetos. Para tal, eram realizadas chamadas com o meu colega durante todo o dia de trabalho.

A planificação das atividades foi sendo definida e adaptada ao longo do estágio pelos orientadores e de acordo com as necessidades dos projetos, não havendo um plano inicial informado. O trabalho foi organizado em projetos para explorar e aplicar determinados conhecimentos técnicos.

5 Trabalho desenvolvido

O trabalho realizado durante o estágio pode ser dividido em duas fases principais: a de formação e pequenos projetos e a do projeto final.

A primeira fase começou logo no primeiro dia de estágio, com a atribuição da minha primeira tarefa: realizar um tutorial sobre MVC em ASP.NET Core, que me permitiu conhecer o C#, .NET e lembrar o MVC. Durante as próximas oito semanas, foi possível ter contacto com outras tecnologias como ASP.NET Core MVC, APIs REST, jQuery, Umbraco CMS, DataTables e Hangfire. Esta fase consistiu numa aprendizagem baseada em tutoriais, seguida de projetos práticos para aplicar os conhecimentos adquiridos.

A segunda fase também teve uma duração aproximada de oito semanas e correspondeu ao desenvolvimento do projeto final. Este projeto foi atribuído pelo segundo orientador na empresa, Renan de Medeiros, que teve um papel importante na definição inicial da solução e no acompanhamento técnico. O projeto revelou-se particularmente desafiante na fase inicial, uma vez que envolveu requisitos técnicos mais exigentes, como a utilização de uma arquitetura em camadas, a aplicação de boas práticas de desenvolvimento e o acesso à base de dados sem recurso a um ORM.

5.1 Formação e Pequenos Projetos

Como referido anteriormente, a primeira fase do estágio teve uma duração aproximada de 8 semanas e foi destinada à adaptação das tecnologias utilizadas no contexto da empresa. Durante este período, foram realizados três tutoriais oficiais e dois pequenos projetos práticos, estes, adaptados ao longo do período, com o objetivo de adquirir bases teóricas.

Os tutoriais serviram como ponto de partida para compreender a estrutura base das aplicações desenvolvidas em ASP.NET Core e em Umbraco. A partir desses conhecimentos, foram depois realizados pequenos projetos que permitiram aplicar os conceitos aprendidos, nomeadamente através da criação de aplicações MVC, APIs, integração com jQuery, desenvolvimento de *websites* em Umbraco, utilização de DataTables, automatização de tarefas e criação de uma extensão para o *backoffice* do Umbraco.

As seguintes subsecções estão ordenadas pela ordem cronológica em que as tarefas foram realizadas, tendo cada uma durado cerca de duas semanas.

5.1.1 Introdução a C#, .NET, MVC e APIs REST

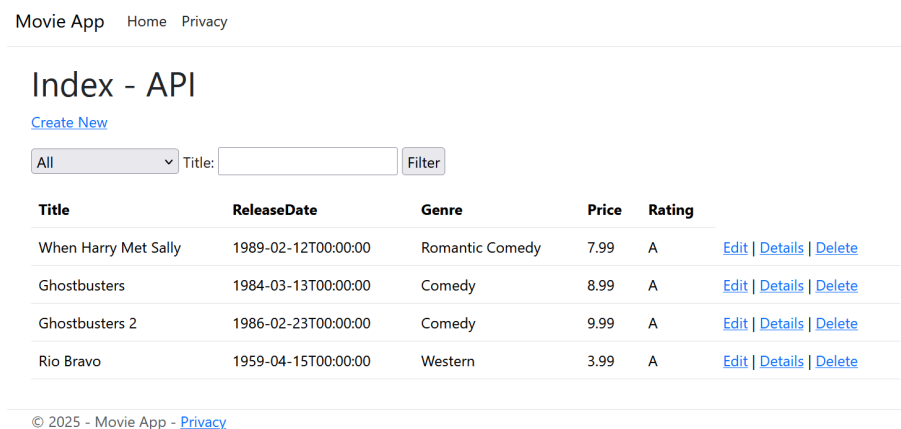
Nas duas primeiras semanas do estágio, o trabalho centrou-se na aprendizagem inicial de C#, .NET e ASP.NET Core. Como não existia experiência prévia com C#, esta fase foi importante para compreender a sintaxe da linguagem, a organização dos projetos em .NET e a forma como uma aplicação *web* é estruturada com ASP.NET Core.

Para introduzir estes conceitos, foram realizados dois tutoriais oficiais da Microsoft. O primeiro incidiu sobre o desenvolvimento de uma aplicação ASP.NET Core MVC, a criação de *controllers*, modelos e *views*, bem como operações CRUD sobre uma entidade [43]. O segundo tutorial focou-se na criação de uma API *controller-based* em ASP.NET Core, permitindo compreender a criação de *endpoints* REST, a utilização de métodos HTTP e a troca de dados em formato JSON [44].

Após a realização destes tutoriais, foi proposto um desafio prático: implementar uma API para o trabalho realizado no primeiro tutorial. Este foi o primeiro projeto, uma aplicação de gestão de filmes, que tinha uma componente MVC responsável pela apresentação das páginas e uma nova componente de API responsável por disponibilizar e manipular os dados.

Na componente MVC foram utilizadas *views* para apresentar as páginas da aplicação, enquanto a API disponibilizava operações para consultar, criar, atualizar e remover filmes. A comunicação entre a interface e a API foi realizada com recurso a jQuery, através de chamadas assíncronas aos *endpoints*.

O projeto utilizou *Entity Framework Core* para o acesso à base de dados *SQL Server*, permitindo persistir os dados dos filmes. Foram também utilizados *Data Transfer Objects* (DTOs) para transferir apenas os dados necessários entre a API e a interface. Esta fase permitiu consolidar conceitos fundamentais como *endpoints*, *controllers*, modelos, *views*, pedidos HTTP, respostas *JavaScript Object Notation* (JSON), operações CRUD e ligação à base de dados. Um excerto do resultado final pode ser visto na Figura 3.



Movie App Home Privacy

Index - API

[Create New](#)

All

Title	ReleaseDate	Genre	Price	Rating	
When Harry Met Sally	1989-02-12T00:00:00	Romantic Comedy	7.99	A	Edit Details Delete
Ghostbusters	1984-03-13T00:00:00	Comedy	8.99	A	Edit Details Delete
Ghostbusters 2	1986-02-23T00:00:00	Comedy	9.99	A	Edit Details Delete
Rio Bravo	1959-04-15T00:00:00	Western	3.99	A	Edit Details Delete

© 2025 - Movie App - [Privacy](#)

Figura 3: Página inicial do primeiro pequeno projeto, a utilizar a API implementada

5.1.2 Desenvolvimento com Umbraco CMS

Após a aprendizagem de C#, .NET, MVC e APIs, o trabalho passou para o desenvolvimento com Umbraco CMS. Inicialmente, foi realizado um tutorial oficial do Umbraco, com o objetivo de compreender o funcionamento do *backoffice*, a criação de tipos de conteúdo, a associação entre conteúdos e templates, e a forma como os dados geridos no CMS são apresentados no *frontend* [45]. Este tutorial foi iniciado na segunda semana e terminado no início da terceira.

Após a realização deste tutorial, foi desenvolvido um pequeno *website* em Umbraco para consolidar o que foi aprendido, cujo resultado final está na Figura 4. Este projeto consistia na recriação da UI de um *website* já existente e fornecido pelo orientador. Desta forma, foi possível explorar com mais detalhe a criação de páginas, a organização da *content tree*, a utilização de templates Razor e a construção de *layouts* reutilizáveis. Foram também usadas *partial views* para dividir a interface em componentes menores, de modo a facilitar a reutilização de código bem como a organização do mesmo.

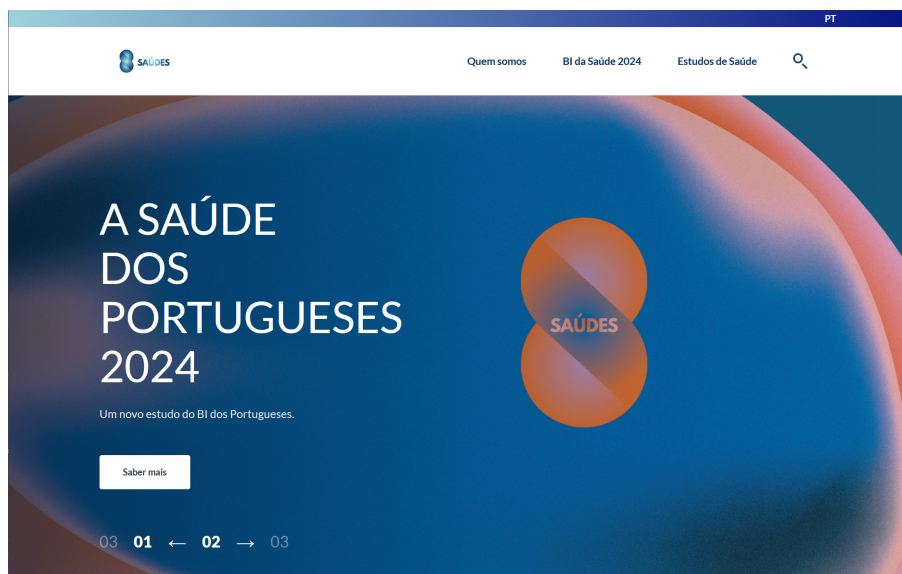


Figura 4: Uma parte da página inicial do *website* realizado em Umbraco

Durante este projeto, foi dada uma atenção especial à organização da aplicação dentro do *backoffice*, Figura 5, porque existem diversas alternativas para representar o mesmo conteúdo. Sendo assim, foi necessário analisar todas estas alternativas e, no final, foi utilizada a alternativa sugerida pelo orientador.

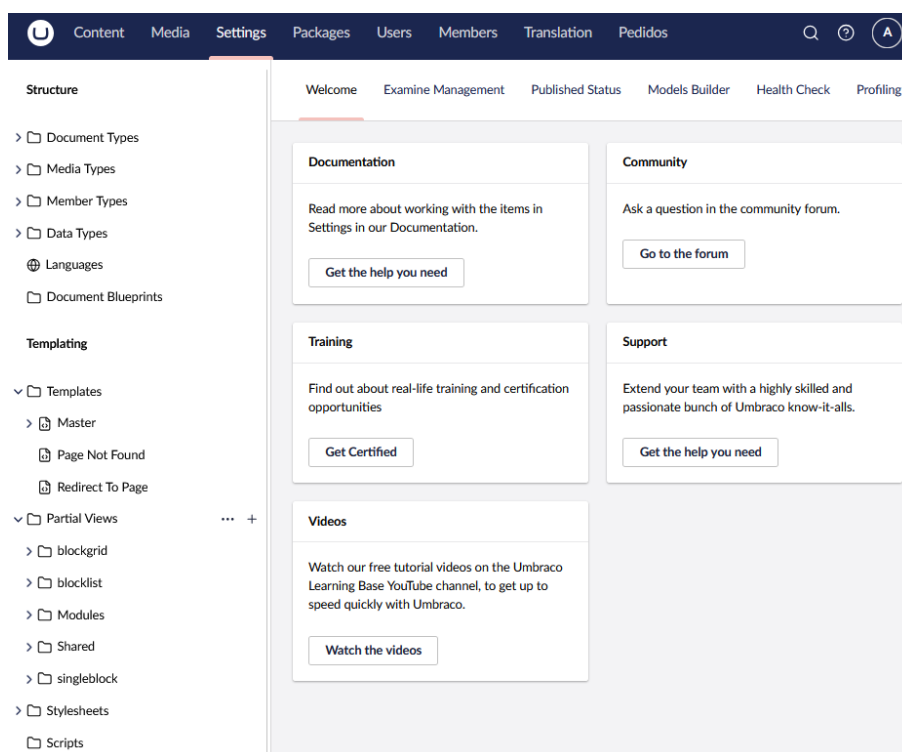


Figura 5: *Backoffice* do Umbraco

Para deixar o *website* o mais parecido possível com a amostra fornecida, foi utilizado o Bootstrap durante a construção, o que facilitou bastante, especialmente ao utilizar componentes já existentes.

5.1.3 Integração com DataTables e automatização de tarefas

Após o desenvolvimento de ambos os projetos e de modo a diversificar a aprendizagem, foram explorados mecanismos de automatização de tarefas e integrações com bibliotecas externas. O objetivo da utilização destes focou-se na melhoria da apresentação dos dados na interface e na experimentação de diferentes formas de executar tarefas automaticamente.

Inicialmente, foi integrada a biblioteca DataTables no primeiro projeto, definido na subsecção 5.1.1, na tabela que lista os filmes. Em vez de apresentar os dados através de uma tabela HTML simples, a biblioteca permitiu adicionar funcionalidades como paginação, pesquisa e ordenação. Desta maneira, foi realizado um *endpoint* específico na API, responsável por receber os parâmetros enviados pela tabela e devolver os dados no formato esperado pela biblioteca.

A comunicação entre a tabela e a API foi realizada através de uma chamada AJAX. Do lado da API os dados eram filtrados, ordenados e paginados antes de serem devolvidos à interface. Ao manter o processamento dos dados no lado do servidor, a tabela passou a carregar apenas os dados necessários para cada página, tornando-a bastante eficiente. O resultado está na Figura 6.

Movie App Home Privacy

Index - API

[Create New](#)

All Title: Filter

10 entries per page Search:

Title	ReleaseDate	Genre	Price	Rating	Actions
Ghostbusters	1984-03-13T00:00:00	Comedy	8.99	A	Edit Details Delete
Ghostbusters 2	1986-02-23T00:00:00	Comedy	9.99	A	Edit Details Delete
Rio Bravo	1959-04-15T00:00:00	Western	3.99	A	Edit Details Delete
When Harry Met Sally	1989-02-12T00:00:00	Romantic Comedy	7.99	A	Edit Details Delete

Showing 1 to 4 of 4 entries

« < 1 > »

© 2025 - Movie App - [Privacy](#)

Figura 6: Página inicial da alteração do primeiro projeto para utilizar DataTables

Para além da integração com DataTables, foram exploradas formas de automatizar tarefas relacionadas com APIs do mesmo projeto. Numa primeira abordagem, foi criada uma aplicação de consola em C# que funcionava como um *script* que fazia um pedido HTTP à API do projeto e guardava o resultado num ficheiro JSON. Esta aplicação foi executada inicialmente no Windows Task Scheduler.

Posteriormente, foi estudado o Hangfire como alternativa ao Windows Task Scheduler. Como o Hangfire executava as tarefas em segundo plano dentro da própria aplicação ASP.NET Core, e não dependia de fatores externos, foi o *software* preferido, com a sua *dashboard* na Figura 7.

Hangfire Dashboard Jobs 0 Retries 0 Recurring Jobs 1 Servers 3 Back to site

Recurring Jobs

Trigger now Delete

Items per page: 10 20 50 100 500

☒	Id	Cron	Time zone	Job	Next execution	Last execution	Created
☒	movies-scheduler	* * * * *	UTC	Scheduler.CreateMovieAsync	in a few seconds	a few seconds ago	5 minutes ago

Total items: 1

Figura 7: Recurring Job da dashboard do Hangfire

5.1.4 Desenvolvimento de uma extensão para o Umbraco

Na última fase dos pequenos projetos, foi desenvolvida uma extensão para o *backoffice* do Umbraco. Esta extensão teve como objetivo integrar dentro do *backoffice* funcionalidades que tinham sido exploradas no primeiro projeto, nomeadamente a apresentação de dados através da biblioteca Datatables e a introdução da exportação dos dados para um ficheiro *Comma Separated Values* (CSV).

Para isso, foi criada uma nova secção no *backoffice* chamada "Pedidos". Esta secção incluía uma *content tree* personalizada e duas *views*: uma para consulta dos dados e outra para exportação.

A primeira *view* apresentava uma tabela com os dados obtidos a partir da API do primeiro projeto, cujo resultado está na Figura 8. Esta integração foi realizada com recurso a componentes Lit e à biblioteca Datatables, permitindo assim reutilizar a lógica de paginação, pesquisa e ordenação já desenvolvida anteriormente, mas agora integrada no *backoffice* do Umbraco.

U Content Media Settings Packages Users Members Translation Pedidos

Meus Pedidos

- Website x
 - Pedidos 1
 - Pedidos 2

Pedidos Export

Dados

10 entries per page Search:

Title	Release Date	Genre	Price	Rating
Ghostbusters	1984-03-13T00:00:00	Comedy	8.99	A
Ghostbusters 2	1986-02-23T00:00:00	Comedy	9.99	A
Rio Bravo	1959-04-15T00:00:00	Western	3.99	A
When Harry Met Sally	1989-02-12T00:00:00	Romantic Comedy	7.99	A

Showing 1 to 4 of 4 entries

Figura 8: Representação da tabela no *backoffice*

A segunda *view* permitia exportar dados para um ficheiro CSV. Para isso, foi necessário criar uma API no primeiro projeto que conseguisse suportar filtros de datas e devolvesse o próprio ficheiro. Esta *view* pode ser vista na Figura 9.

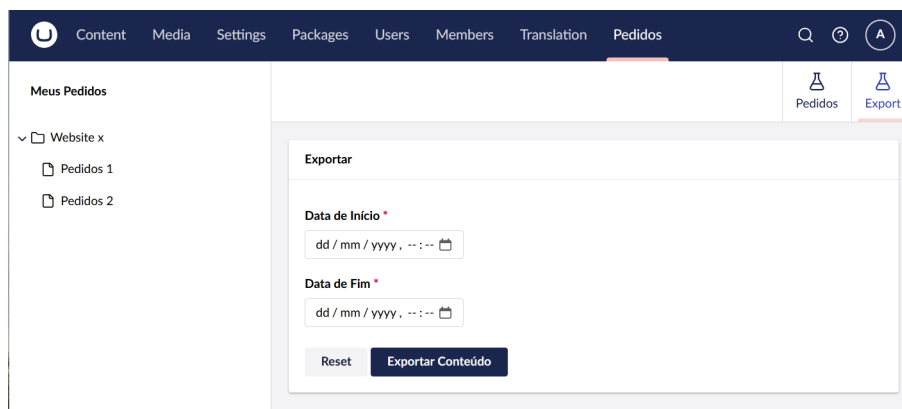


Figura 9: Exportação de dados

5.2 Projeto Final

Após a conclusão da fase de formação, iniciou-se a segunda fase do estágio, dedicada ao desenvolvimento de um projeto de maior dimensão.

Este projeto teve uma duração aproximada de oito semanas e consistiu numa aplicação *web* para entrega de refeições. Esta aplicação foi pensada para dois tipos de utilizadores: clientes e fornecedores.

Ao contrário dos projetos desenvolvidos anteriormente, este projeto exigiu uma organização mais cuidada da solução, conforme solicitado pelo tutor Renan. Desta forma, foi utilizada uma arquitetura em camadas, de modo a separar o domínio, a lógica aplicacional, a infraestrutura, a API e a apresentação. Esta separação teve como objetivo tornar o projeto mais organizado, facilitar a manutenção do código e aplicar boas práticas de desenvolvimento.

Esta fase introduziu também conceitos novos e mais técnicos, como a autenticação e autorização com JWT, o acesso à base de dados sem recurso a um ORM e o desenvolvimento de interfaces com Blazor WebAssembly e MudBlazor.

As seguintes subsecções estão ordenadas pela ordem cronológica em que as tarefas foram realizadas, apesar de terem sido realizadas melhorias e alterações durante todo o desenvolvimento do projeto.

5.2.1 Planeamento inicial, princípios SOLID e arquitetura

Antes de começar o projeto final, foi dedicada uma semana à aprendizagem de princípios SOLID e de boas práticas de programação. Esta semana foi importante para compreender a organização de uma aplicação de maior dimensão, separar responsabilidades e deixar o código pronto para manter e evoluir.

Na proposta inicial foram definidos alguns requisitos técnicos importantes. A aplicação deveria ser organizada em camadas, aplicar os princípios SOLID, validar os dados introduzidos e aceder à base de dados sem recorrer a um ORM, utilizando ADO.NET [12]. Foi também definido que a autenticação deveria ser realizada com JWT, incluindo *login* com utilizador e palavra-passe, armazenamento seguro da palavra-passe, entre outros.

Com base nestes requisitos, foi planeada uma solução dividida em vários projetos para separar as responsabilidades da aplicação. Esta divisão foi inicialmente planeada em conjunto com o colega de estágio e com o orientador que ajudou bastante a organizar os projetos no IDE e a planejar outros detalhes importantes, como as entidades e a base de dados que será referida na subsecção 5.2.2.

Relativamente à camada de apresentação, a escolha da tecnologia ficou em aberto na proposta inicial. No entanto, foi recomendada a utilização de Blazor, juntamente com a biblioteca MudBlazor, que fornecia componentes reutilizáveis. Esta recomendação do orientador acabou por ser a escolhida e utilizada, por ser uma tecnologia atual e por já ter estes componentes.

Assim, a camada de apresentação foi dividida em duas aplicações Blazor WebAssembly, uma para clientes e outra para fornecedores. A camada API ficou responsável por expor as funcionalidades da aplicação através de *endpoints* HTTP. A camada de aplicação ficou responsável pela lógica principal do sistema. A camada de domínio ficou responsável pelas entidades e regras principais da aplicação. Por fim, a camada de infraestrutura ficou responsável pelo acesso à base de dados através de ADO.NET.

5.2.2 Camadas *Domain* e *Infrastructure*

Após a definição inicial da arquitetura e das tecnologias, o desenvolvimento começou pelas camadas mais inferiores, a *Infrastructure*, em conjunto com a *Domain*.

Na camada *Infrastructure* foi implementado o acesso à base de dados SQL Server através do ADO.NET, com recurso a `Microsoft.Data.SqlClient`. Ao contrário da utilização de um ORM, esta abordagem exigiu a criação manual de instruções SQL, dos parâmetros, a leitura dos resultados, bem como o *mapping* das entidades.

Desta forma, foram criadas as tabelas, definidas as suas chaves primárias e estrangeiras e implementados os repositórios responsáveis pelo acesso dos dados. Estes repositórios incluíam sempre interfaces e realizavam operações CRUD, utilizando comandos SQL parametrizados.

Para atualizar automaticamente alguns campos (e.g. `UpdatedAt`), foi também implementado um mecanismo com *triggers*, o que deixou o código mais limpo nos repositórios.

Relativamente à camada *Domain*, esta incluiu os principais conceitos da aplicação (e.g. utilizadores, produtos) e enumerações para representar estados (e.g. estado do pagamento), com as respetivas validações.

Por fim, estas camadas foram bastante úteis para a realização das próximas da arquitetura porque representam a base de todo o projeto, sendo esta a principal razão de ter sido começado por aqui.

5.2.3 Camada *Application* e Autenticação

Nesta camada foram criados serviços para as principais funcionalidades do sistema, como autenticação, produtos, *stock*, entre outras. Estes serviços representavam a lógica de "negócio" da aplicação, recebiam os dados vindos da API, aplicavam validações, acediam aos repositórios e devolviam a resposta para a camada seguinte.

Uma das primeiras funcionalidades implementadas nesta camada foi a autenticação. Para isso, foi ne-

cessário ter o cuidado de armazenar palavras-passe de uma forma segura, através de *hashing*, evitando guardar a palavra-passe original na base de dados. Após um *login* bem-sucedido, era gerado um *token* JWT, que depois seria utilizado nos pedidos seguintes para identificar o utilizador autenticado.

Para distinguir o tipo de utilizador e controlar o acesso às funcionalidades, o *token* gerado incluía dados não sensíveis, como o ID do utilizador e o seu papel na aplicação, distinguindo clientes de fornecedores. Estes dados permitiam que a API aplicasse regras de autorização de acordo com o tipo de utilizador autenticado. Foi também criado um serviço responsável por renovar este *token*, a partir de um *token* anterior válido.

Para além da autenticação, foram implementadas as regras associadas aos produtos, *stock*, carrinho de compras e encomendas. No caso dos produtos, a criação de um novo produto implicava também a criação do respetivo *stock* inicial e de um movimento de *stock*.

A gestão das encomendas foi uma das partes mais importantes desta camada, por envolver várias validações e alterações relacionadas. No processo de *checkout*, era necessário validar o carrinho ativo do cliente, verificar a existência de *stock*, atualizar as quantidades disponíveis, registar movimentos de *stock*, fechar o carrinho e criar a encomenda. Para garantir a consistência dos dados nestas operações, foram utilizadas *transactions*, de modo que, caso uma das etapas falhasse, as restantes alterações não fossem confirmadas [13].

Para seguir com as boas práticas estudadas anteriormente, foi ainda criado um mecanismo de resposta com padrões. Este mecanismo permitiu devolver de forma consistente os resultados de sucesso, erros de validação, recursos não encontrados, acessos proibidos ou não autorizados, entre outros, para facilitar a ligação com a camada da API.

5.2.4 Camada API

Depois da implementação da camada *Application*, foi desenvolvida a camada *API*, responsável por expor as funcionalidades da aplicação através de *endpoints* HTTP e comunicar entre as aplicações de *presentation* e a lógica das camadas internas.

Nesta camada foram criados *controllers* para as principais áreas da aplicação, que serviam para encaminhar os pedidos HTTP recebidos para o respetivo serviço da camada *Application* e devolver a resposta adequada ao cliente.

Esta API também implementou autenticação e autorização com JWT. Desta forma, algumas rotas ficaram protegidas e apenas podiam ser acedidas por utilizadores autenticados. Foram também aplicadas algumas regras de autorização, conforme o tipo de utilizador, distinguindo as funcionalidades disponíveis para clientes e fornecedores.

Para deixar as respostas mais consistentes e uniformes, os *controllers* utilizavam uma resposta base, que convertia os resultados vindos da camada *Application* em respostas HTTP adequadas.

Nesta camada também foi dada a devida importância a alguns elementos importantes, como o CORS, Swagger, autenticação JWT e *middlewares*. Estes *middlewares* foram utilizados para tratar erros globais, obter informação do utilizador autenticado e renovar o *token*.

Durante o desenvolvimento, os *endpoints* foram testados com recurso ao Postman e ao Swagger, para

verificar se os pedidos, respostas e funcionalidades estavam bem implementados. Após o desenvolvimento, foram novamente testados para validar o *backend* antes de iniciar a integração com a camada seguinte.

5.2.5 Camada *Presentation*

Após a conclusão da parte principal do *backend*, iniciou-se a camada *Presentation*, que correspondeu ao *frontend* da aplicação. Esta camada é responsável não só por mostrar o conteúdo da página, como também por comunicar com a API através de serviços HTTP responsáveis por enviar pedidos e tratar as respostas recebidas.

Esta camada foi dividida em duas aplicações distintas, a destinada aos clientes e a destinada aos fornecedores. O desenvolvimento desta camada foi dividido entre mim e o meu colega de estágio. A aplicação destinada ao fornecedor ficou principalmente da minha responsabilidade, enquanto a destinada ao cliente foi da responsabilidade do meu colega. Apesar desta divisão, houve colaboração em ambas ao longo do desenvolvimento, especialmente na criação de componentes partilhados.

Na aplicação do fornecedor, foram implementadas funcionalidades como autenticação, *dashboard* (Figura 11), gestão de perfil, gestão de produtos (Figura 10), acompanhamento de encomendas (Figura 12) e de *stock*. As restantes imagens encontram-se no Anexo A.

Para evitar a duplicação de código e cumprir com as boas práticas, foi ainda criado um projeto auxiliar dentro desta camada para partilhar lógica comum entre as aplicações. Dentro deste projeto eram colocados sobretudo componentes, serviços e autenticação, porque eram comuns a ambos os projetos.

The screenshot displays the 'Products' management interface of the FoodApp. The top navigation bar includes links for Dashboard, Products (active), Orders, Stock Movements, Profile, and Logout. The main content area is titled 'Products' and includes a sub-header 'Create, update, and remove your product offerings.' and a 'REFRESH' button. On the left, there is an 'Edit product' form with fields for Name (Limonada Caseira), Description (Limonada fresca feita no momento.), Image URL (https://external-content.duckduckgo.com/i...), Price (EUR 2.50), and Stock (55). The form has 'CANCEL' and 'SAVE CHANGES' buttons. On the right, there is a table of products with columns for Product, Price, Stock, and Updated. The table lists seven products: Agua Mineral, Limonada Caseira, Cheesecake de Frutos Vermelhos, Mousse de Chocolate, Salada Caesar, Risotto de Cogumelos, and Hamburguer Artesanal. Each product row includes a small image, the product name, a brief description, the price in EUR, the stock level (indicated by a green circle with a number), the last updated date and time, and edit/delete icons.

Product	Price	Stock	Updated
Agua Mineral Garrafa de agua mineral 50cl.	EUR 1.50	79	19/06/2026 11:18
Limonada Caseira Limonada fresca feita no momento.	EUR 2.50	55	19/06/2026 11:18
Cheesecake de Frutos Vermelhos Cheesecake cremoso com cobertura de frutos vermelhos.	EUR 4.50	38	19/06/2026 11:19
Mousse de Chocolate Mousse caseira de chocolate negro.	EUR 3.90	49	19/06/2026 11:19
Salada Caesar Alface romana, frango grelhado, croutons, parmesao e molho Caesar.	EUR 8.60	29	19/06/2026 11:19
Risotto de Cogumelos Risotto cremoso com cogumelos frescos e parmesao.	EUR 11.80	28	19/06/2026 11:20
Hamburguer Artesanal Hamburguer de vaca com queijo, bacon, alface e molho da casa.	EUR 10.95	42	19/06/2026 11:20

Figura 10: Pannel de produtos e edição dos mesmos

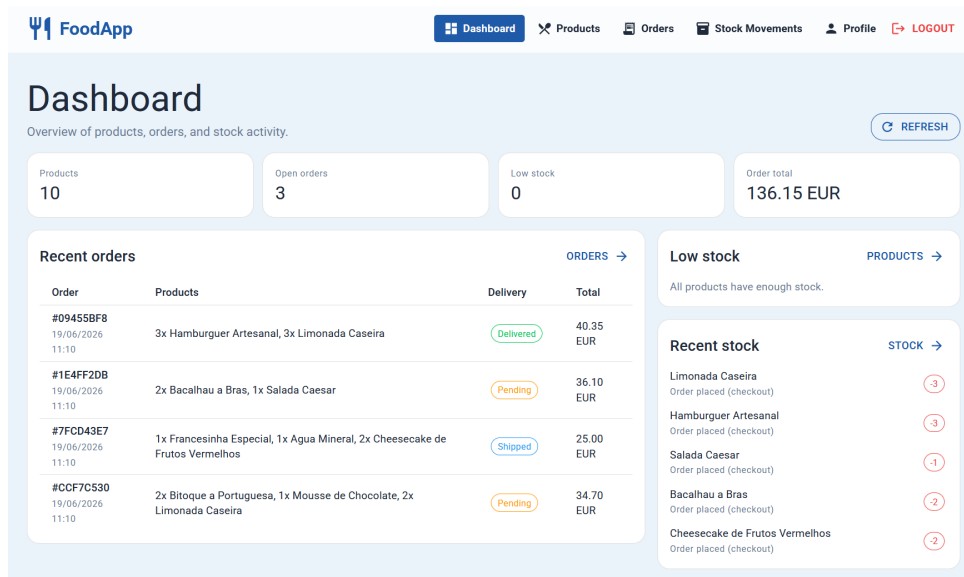


Figura 11: Dashboard do fornecedor

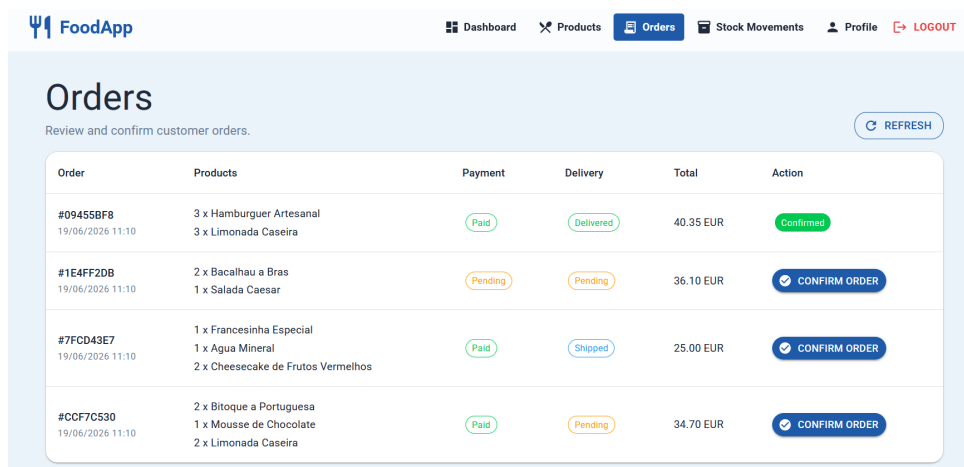


Figura 12: Encomendas do fornecedor

5.2.6 Testes finais, correções e validação

Após terminar todas as camadas da aplicação, foram realizados testes de forma manual, através da utilização da interface desenvolvida. Assim, foi possível testar os fluxos da aplicação e encontrar alguns *bugs*, nomeadamente na autenticação, que rapidamente foram resolvidos. Para além disso, foram também corrigidos os tempos de expiração dos *tokens* JWT, porque a aplicação saiu da fase de desenvolvimento.

Após a conclusão dos testes e correções, foi realizada uma reunião final com o orientador Renan de Medeiros. Nesta reunião, apresentou-se o projeto final desenvolvido, foram explicadas as principais funcionalidades e discutido o resultado obtido. No final, foi também gravada uma apresentação do projeto, com o objetivo de demonstrar o trabalho realizado à Plexus Tech.

6 Avaliação crítica

O estágio curricular permitiu-me compreender melhor a realidade do desenvolvimento de *software*. Ao longo das 240 horas de estágio, tive a oportunidade de aprender novas tecnologias, compreender melhor o funcionamento de uma empresa e aplicar conhecimentos adquiridos ao longo do curso num contexto mais prático e próximo da realidade profissional.

A integração na empresa foi muito positiva desde o início. Fui bem recebido pela empresa e senti que os orientadores, Ricardo Godinho e Renan de Medeiros, tinham uma grande disponibilidade para esclarecer dúvidas e partilhar conhecimento.

Antes do estágio, não tinha experiência com a maioria das tecnologias utilizadas, o que tornou a fase inicial mais desafiadora. Ao longo do estágio fui adquirindo conhecimento não só nestas tecnologias como na resolução de problemas, na correção de erros, na minha comunicação e no trabalho em equipa.

Apesar da experiência ter sido bastante positiva, considero que a empresa poderia ter elaborado um plano de estágio. Em alguns momentos, a ausência de um plano levou a períodos sem tarefas a realizar. No entanto, esses períodos foram aproveitados para melhorar projetos anteriores, estudar temas relacionados com a área e aprofundar as boas práticas do desenvolvimento.

Outro aspeto negativo que considero relevante é o facto de não ter trabalhado diretamente num projeto interno da empresa, como tinha sido definido nos objetivos iniciais, nem ter sido integrado numa equipa de desenvolvimento. Ainda assim, considero que o projeto final cumpriu um papel importante, pois permitiu simular desafios comuns no desenvolvimento de uma aplicação *web* moderna, desde a organização da arquitetura até à implementação e integração das diferentes camadas.

De forma geral, considero que o estágio cumpriu as minhas expectativas e contribuiu de forma significativa para a minha preparação para o mercado de trabalho.

Referências

- [1] DigitalWorks, “Digitalworks – digital transformation.” Disponível em: <https://pt.linkedin.com/company/digitalworks---engineering-creativity>. Consultado em: 15 de junho de 2026.
- [2] DigitalWorks, “A plexus tech integrou no seu grupo a digitalworks.” Disponível em: https://www.linkedin.com/posts/digitalworks---engineering-creativity_somosplexus-weareplexus-activity-7191048512879419392-TdCl, 2024. Consultado em: 15 de junho de 2026.
- [3] Plexus Tech, “Somos.” Disponível em: <https://www.plexus.es/pt-pt/somos/>. Consultado em: 15 de junho de 2026.
- [4] Plexus Tech, “Contacto e localizações.” Disponível em: <https://www.plexus.es/contacto/>. Consultado em: 15 de junho de 2026.
- [5] MDN Web Docs, “Overview of http.” Disponível em: <https://developer.mozilla.org/docs/Web/HTTP/Guides/Overview>. Consultado em: 18 de junho de 2026.
- [6] Microsoft Corporation, “Overview of asp.net core mvc.” Disponível em: <https://learn.microsoft.com/aspnet/core/mvc/overview>. Consultado em: 18 de junho de 2026.
- [7] Microsoft Corporation, “Common web application architectures.” Disponível em: <https://learn.microsoft.com/dotnet/architecture/modern-web-apps-azure/common-web-application-architectures#clean-architecture>. Consultado em: 18 de junho de 2026.
- [8] Microsoft Corporation, “Use domain analysis to model microservices.” Disponível em: <https://learn.microsoft.com/azure/architecture/microservices/model/domain-analysis>. Consultado em: 18 de junho de 2026.
- [9] Microsoft Corporation, “C# best practices: Dangers of violating solid principles in c#.” Disponível em: <https://learn.microsoft.com//archive/msdn-magazine/2014/may/csharp-best-practices-dangers-of-violating-solid-principles-in-csharp>, 2014. Consultado em: 18 de junho de 2026.
- [10] Oracle, “What is a relational database?” Disponível em: <https://www.oracle.com/database/what-is-a-relational-database/>, 2021. Consultado em: 18 de junho de 2026.
- [11] Microsoft Corporation, “Entity framework core documentation.” Disponível em: <https://learn.microsoft.com/ef/core/>. Consultado em: 17 de junho de 2026.
- [12] Microsoft Corporation, “Ado.net documentation.” Disponível em: <https://learn.microsoft.com/dotnet/framework/data/adonet/>. Consultado em: 17 de junho de 2026.
- [13] Microsoft Corporation, “Local transactions.” Disponível em: <https://learn.microsoft.com/dotnet/framework/data/adonet/local-transactions>. Consultado em: 18 de junho de 2026.
- [14] Jones, Michael and Bradley, John and Sakimura, Nat, “Json web token (jwt).” Disponível em: <https://datatracker.ietf.org/doc/html/rfc7519>. Consultado em: 19 de junho de 2026.

-
- [15] Microsoft Corporation, “Configure jwt bearer authentication in asp.net core.” Disponível em: <https://learn.microsoft.com/aspnet/core/security/authentication/configure-jwt-bearer-authentication>. Consultado em: 19 de junho de 2026.
- [16] Microsoft Corporation, “Role-based authorization in asp.net core.” Disponível em: <https://learn.microsoft.com/aspnet/core/security/authorization/roles>. Consultado em: 19 de junho de 2026.
- [17] Hardt, Dick, “The oauth 2.0 authorization framework (rfc 6749).” Disponível em: <https://datatracker.ietf.org/doc/html/rfc6749>, 2012. Consultado em: 19 de junho de 2026.
- [18] Microsoft Corporation, “Windows operating system documentation.” Disponível em: <https://www.microsoft.com/windows>. Consultado em: 17 de junho de 2026.
- [19] Microsoft Corporation, “C# programming guide.” Disponível em: <https://learn.microsoft.com/dotnet/csharp/>. Consultado em: 17 de junho de 2026.
- [20] Microsoft Corporation, “.net sdk documentation.” Disponível em: <https://learn.microsoft.com/dotnet/core/sdk/>. Consultado em: 17 de junho de 2026.
- [21] Microsoft Corporation, “Asp.net core documentation.” Disponível em: <https://learn.microsoft.com/aspnet/core/>. Consultado em: 17 de junho de 2026.
- [22] Microsoft Corporation, “Sql server technical documentation.” Disponível em: <https://learn.microsoft.com/sql/sql-server/>. Consultado em: 17 de junho de 2026.
- [23] Microsoft Corporation, “Sql server management studio (ssms).” Disponível em: <https://learn.microsoft.com/sql/ssms/>. Consultado em: 17 de junho de 2026.
- [24] Umbraco HQ, “Umbraco cms documentation.” Disponível em: <https://docs.umbraco.com/>. Consultado em: 17 de junho de 2026.
- [25] Microsoft Corporation, “Blazor documentation.” Disponível em: <https://learn.microsoft.com/en-us/aspnet/core/blazor/>. Consultado em: 17 de junho de 2026.
- [26] World Wide Web Consortium (W3C), “Webassembly specification.” Disponível em: <https://webassembly.org/>. Consultado em: 17 de junho de 2026.
- [27] Microsoft Corporation, “Razor syntax reference.” Disponível em: <https://learn.microsoft.com/aspnet/core/mvc/views/razor>. Consultado em: 17 de junho de 2026.
- [28] Bootstrap Team, “Bootstrap documentation.” Disponível em: <https://getbootstrap.com/docs/>. Consultado em: 17 de junho de 2026.
- [29] MudBlazor Contributors, “Mudblazor documentation.” Disponível em: <https://mudblazor.com/>. Consultado em: 17 de junho de 2026.
- [30] Postman Inc., “Postman learning center.” Disponível em: <https://learning.postman.com/>. Consultado em: 17 de junho de 2026.
- [31] SmartBear Software, “Swagger documentation.” Disponível em: <https://swagger.io/docs/>. Consultado em: 17 de junho de 2026.
-

-
- [32] OpenAPI Initiative, “Openapi specification.” Disponível em: <https://www.openapis.org/>. Consultado em: 17 de junho de 2026.
- [33] SpryMedia Ltd, “Datatables documentation.” Disponível em: <https://datatables.net/>. Consultado em: 17 de junho de 2026.
- [34] Hangfire.io, “Hangfire documentation.” Disponível em: <https://www.hangfire.io/>. Consultado em: 17 de junho de 2026.
- [35] Microsoft Corporation, “Windows task scheduler documentation.” Disponível em: <https://learn.microsoft.com/windows/win32/taskschd/task-scheduler-start-page>. Consultado em: 17 de junho de 2026.
- [36] Software Freedom Conservancy, “Git documentation.” Disponível em: <https://git-scm.com/doc>. Consultado em: 17 de junho de 2026.
- [37] GitHub Inc., “Github documentation.” Disponível em: <https://docs.github.com/>. Consultado em: 17 de junho de 2026.
- [38] Microsoft Corporation, “Microsoft edge.” Disponível em: <https://www.microsoft.com/edge>. Consultado em: 17 de junho de 2026.
- [39] Mozilla Foundation, “Mozilla firefox.” Disponível em: <https://www.mozilla.org/firefox/>. Consultado em: 17 de junho de 2026.
- [40] Microsoft Corporation, “Nuget documentation.” Disponível em: <https://learn.microsoft.com/nuget/>. Consultado em: 17 de junho de 2026.
- [41] Plexus Tech, “Plextime.” Referência interna da Plexus Tech. Consultado em: 17 de junho de 2026.
- [42] Redmine, “Redmine project management software.” Disponível em: <https://www.redmine.org/>. Consultado em: 17 de junho de 2026.
- [43] Microsoft, “Get started with asp.net core mvc.” Disponível em: <https://learn.microsoft.com/en-us/aspnet/core/tutorials/first-mvc-app/start-mvc>. Consultado em: 16 de junho de 2026.
- [44] Microsoft, “Tutorial: Create a controller-based web api with asp.net core.” Disponível em: <https://learn.microsoft.com/en-us/aspnet/core/tutorials/first-web-api>. Consultado em: 16 de junho de 2026.
- [45] Umbraco, “Getting started - creating a basic website.” Disponível em: <https://docs.umbraco.com/umbraco-cms/develop-with-umbraco/tutorials/creating-a-basic-website/getting-started>. Consultado em: 16 de junho de 2026.

Anexo A - Imagens do projeto final

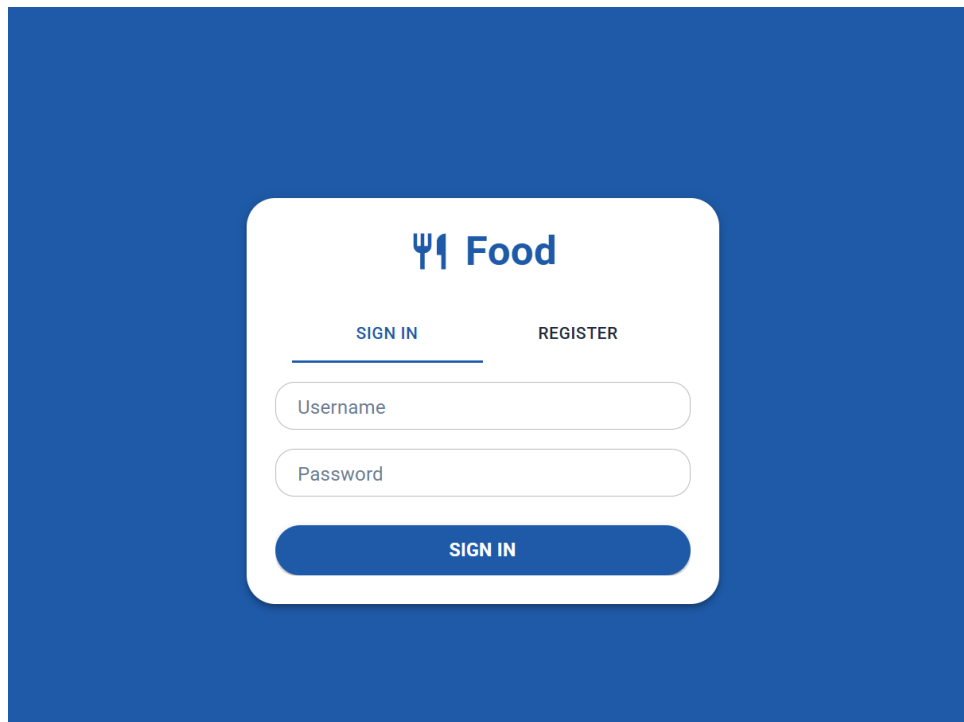


Figura 13: Login do fornecedor

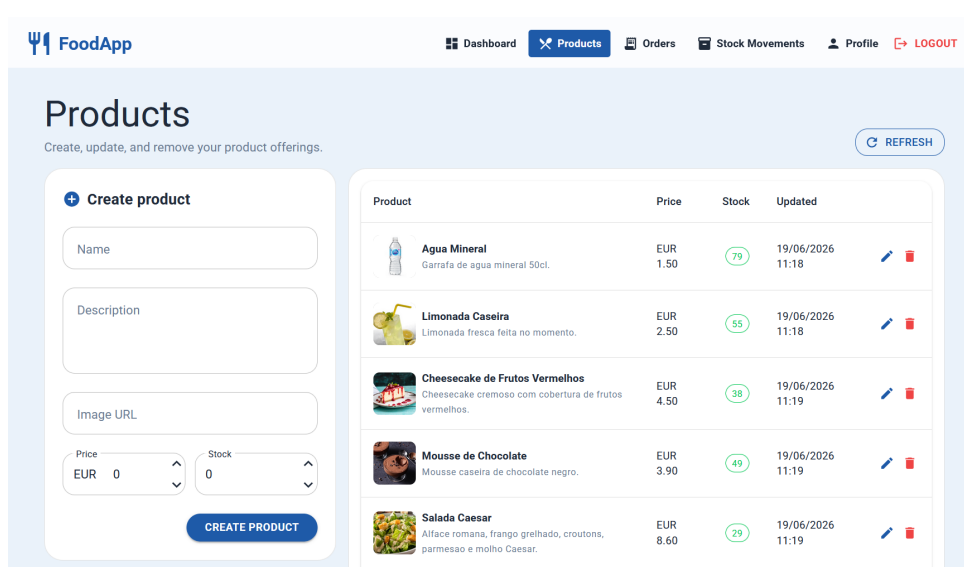


Figura 14: Painel de produtos e criação dos mesmos

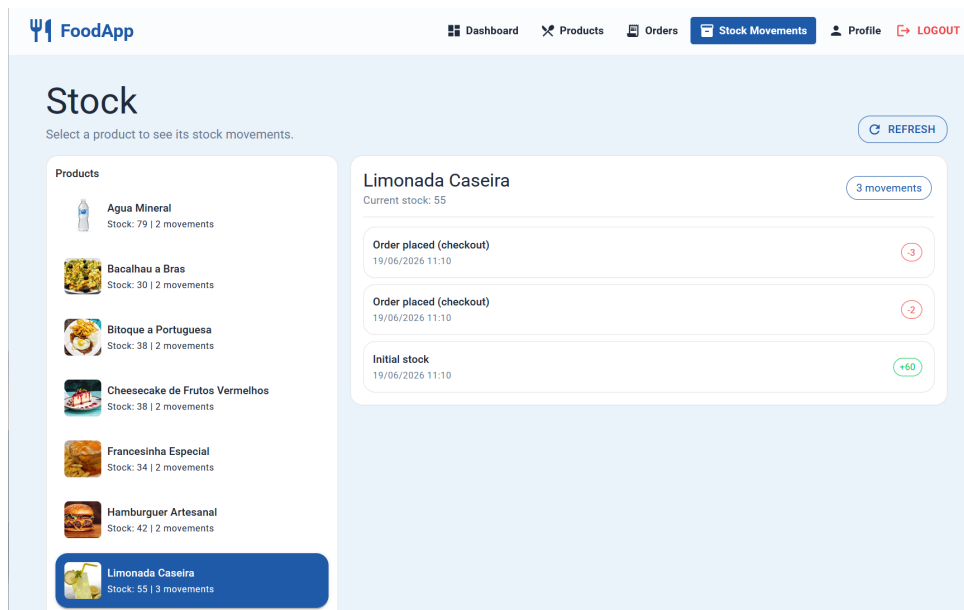


Figura 15: Stock para os produtos do fornecedor

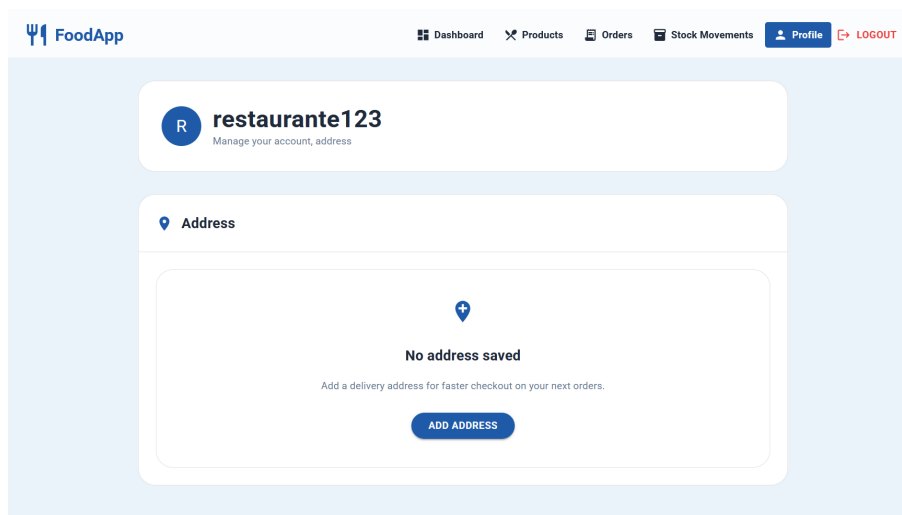


Figura 16: Perfil sem endereço

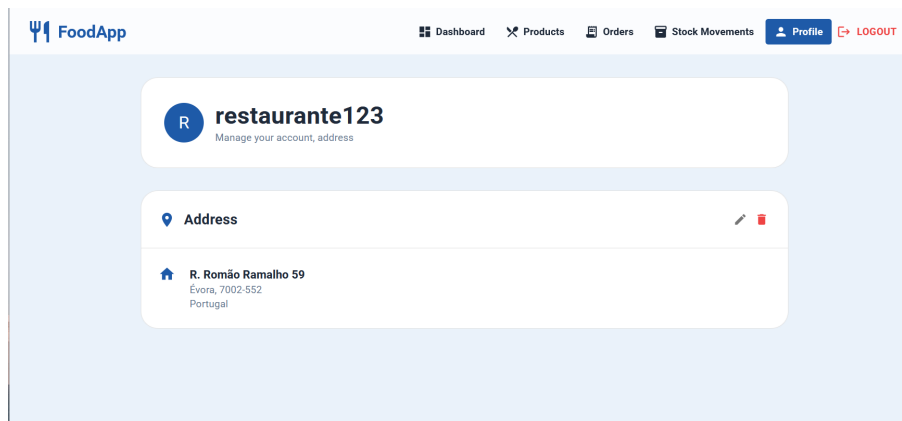


Figura 17: Perfil com endereço